

Universidad Católica Boliviana
“San Pablo”
Ingeniería de Software
Sistema de gestión para una librería



Docente: Ing. Pacheco Arteaga Miguel Angel

Elaborado por:

- Arce Chiri Alejandro Gabriel
- Pattzy Gomez Diana Tatiana

Contenido

1.	Información general.....	1
2.	Introducción.....	1
3.	Objetivos de la arquitectura.....	1
4.	C1-Diagrama de contexto.....	1
5.	C2-Diagrama de Contenedores	2
6.	Flujo general de comunicación.....	3
7.	C3-Diagrama de componentes.....	4
8.	ADR-001 – Uso de una API REST centralizada	5
9.	ADR-002 SQL Server como persistencia y Redis como cache.....	6
10.	Relacion entre C4 y los requisitos	7

1. Información general

Proyecto: Sistema de Gestión para Librería

Documento: Arquitectura de Software

Versión: 1.0

Modelo: C4

Tecnologías: React, Flutter, ASP.NET Core 8, SQL Server y Redis

Estándar de referencia: Modelo C4 para documentación de arquitectura

2. Introducción

El presente documento describe la arquitectura propuesta para el Sistema de Gestión de la Librería utilizando el modelo C4, considerando los niveles de **Contexto**, **Contenedores** y **Componentes**.

La arquitectura busca representar de manera progresiva cómo interactúan los usuarios con el sistema, qué tecnologías conforman la solución y cómo se organiza internamente el backend.

El sistema estará orientado a centralizar la gestión de productos, inventario, compras, pedidos, clientes y servicios de maquetas, procesos que actualmente se realizan mediante registros manuales, control visual del stock y comunicación por WhatsApp.

3. Objetivos de la arquitectura

La arquitectura propuesta tiene los siguientes objetivos:

- Centralizar la información de la librería.
- Separar la interfaz de usuario de la lógica de negocio.
- Permitir el acceso mediante una aplicación web y una aplicación móvil.
- Mantener la información persistente en SQL Server.
- Utilizar Redis para información que requiera acceso frecuente.
- Proteger el acceso mediante autenticación basada en JWT.
- Facilitar el mantenimiento y evolución del sistema.
- Mantener una arquitectura que permita incorporar nuevas funcionalidades posteriormente.

4. C1-Diagrama de contexto

El nivel C1 representa el sistema desde una perspectiva general. Se muestran los principales usuarios y sistemas externos que interactúan con el Sistema de Gestión de la Librería.

Actores

Cliente

- Consulta productos.
- Solicita productos.
- Realiza pedidos.
- Solicita servicios de maquetas.

Personal de la librería

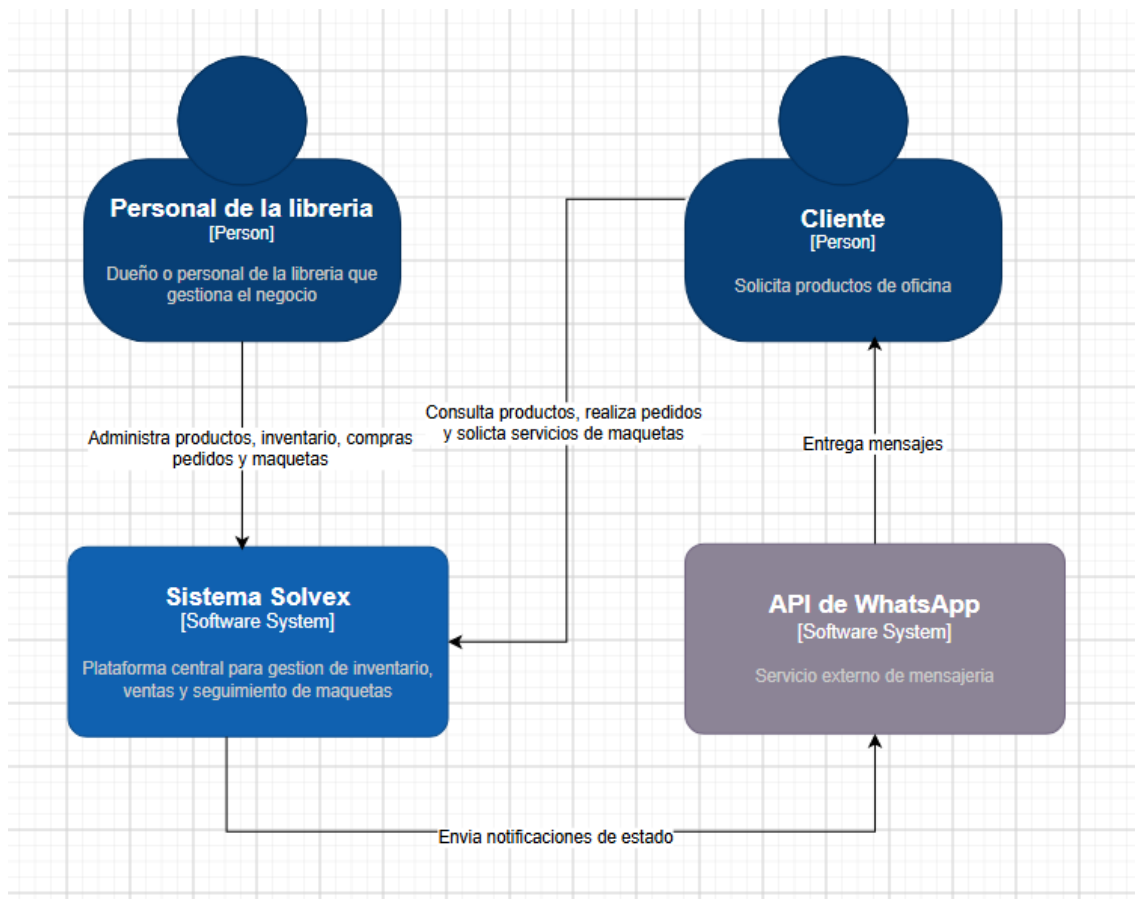
- Administra productos.
- Controla inventario.
- Registra compras.
- Gestiona pedidos.

- Registra y actualiza maquetas.
- Consulta información de clientes.

WhatsApp

- Actualmente es utilizado como canal mediante el cual los clientes envían consultas y listas de útiles.

La integración directa con WhatsApp no se considera una funcionalidad confirmada todavía; por ello se representa como un sistema externo del contexto y no como una integración obligatoria.



Explicación

El sistema constituye el elemento central del contexto. Los clientes y el personal de la librería interactúan con él mediante las aplicaciones desarrolladas para la solución. WhatsApp se representa como un sistema externo debido a que actualmente forma parte del proceso de atención de pedidos y listas de útiles, aunque todavía no se ha confirmado una integración automática con la plataforma.

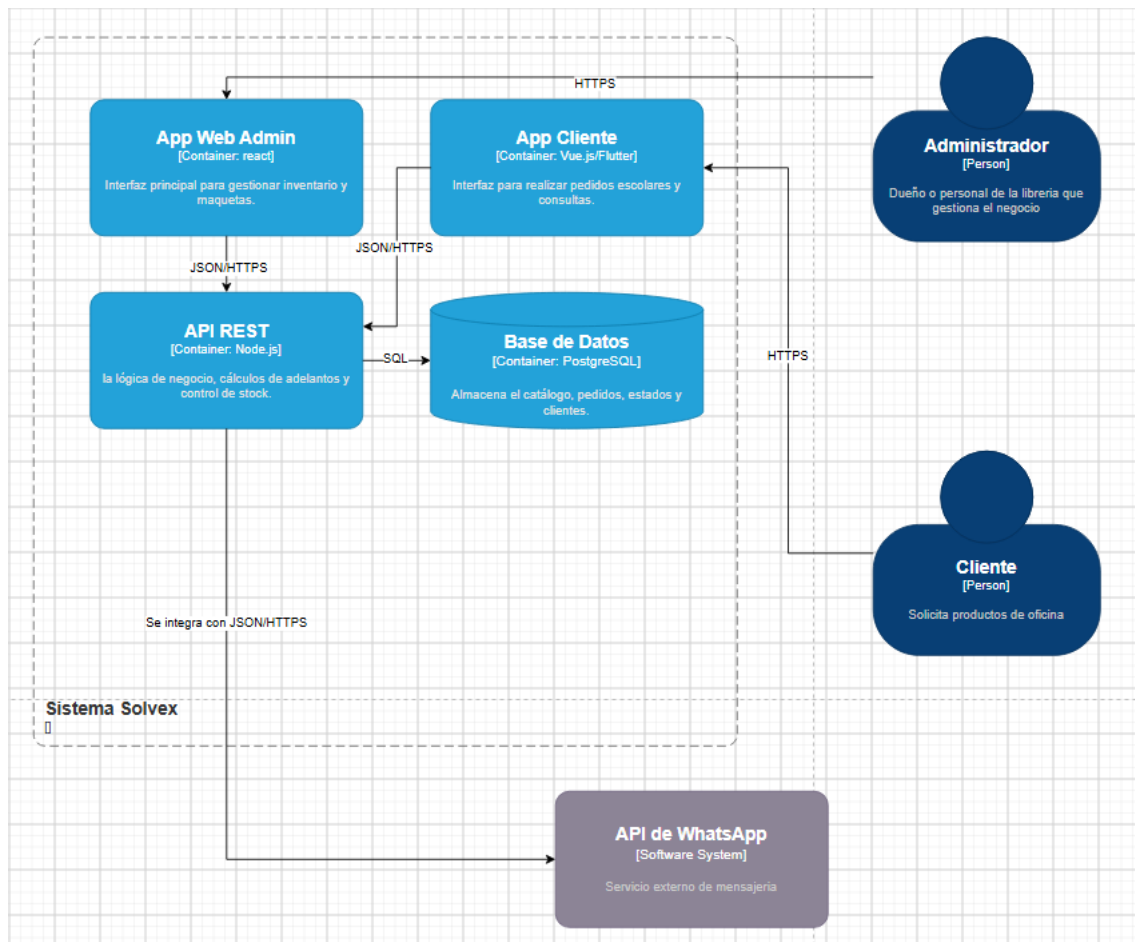
5. C2-Diagrama de Contenedores

El nivel C2 descompone el sistema en sus principales contenedores tecnológicos. La solución estará compuesta por:

1. Aplicación Web.
2. Aplicación móvil.
3. API REST.
4. Base de datos SQL Server.
5. Caché Redis.

5.1 Contenedores

ID	Contenedor	Tecnología	Responsabilidad
C2-01	Aplicación Web	React	Interfaz para clientes y personal de la librería
C2-02	Aplicación móvil	Flutter	Acceso móvil al sistema
C2-03	API REST	ASP.NET Core 8	Lógica de negocio y servicios del sistema
C2-04	Base de datos	SQL Server	Persistencia de la información
C2-05	Caché	Redis	Almacenamiento temporal de información consultada frecuentemente



6. Flujo general de comunicación

La comunicación principal será:

Usuario

|

| HTTPS



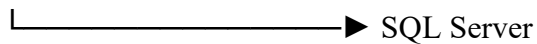
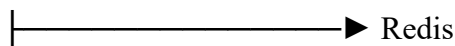
React / Flutter



REST + JSON



ASP.NET Core 8 API



Protocolo

Comunicación	Protocolo
Usuario → React	HTTPS
Usuario → Flutter	HTTPS
React → API	REST / HTTPS / JSON
Flutter → API	REST / HTTPS / JSON
API → SQL Server	SQL
API → Redis	Redis Protocol

7. C3-Diagrama de componentes

El nivel C3 muestra la estructura interna del backend.

Se utilizará una arquitectura organizada en:

Controllers → Services → Repositories → SQL Server

Además, se incorporarán componentes transversales como el **Middleware JWT** y adaptadores para servicios externos.

7.1 Componentes principales

Controllers

Reciben las solicitudes HTTP y las dirigen hacia los servicios correspondientes.

- ProductController
- InventoryController
- SupplierController
- PurchaseController
- OrderController
- CustomerController
- ModelController
- PaymentController

Services

Contienen las reglas de negocio.

- ProductService
- InventoryService
- SupplierService

- PurchaseService
- OrderService
- CustomerService
- ModelService
- PaymentService

Repositories

Gestionan el acceso a la información persistida.

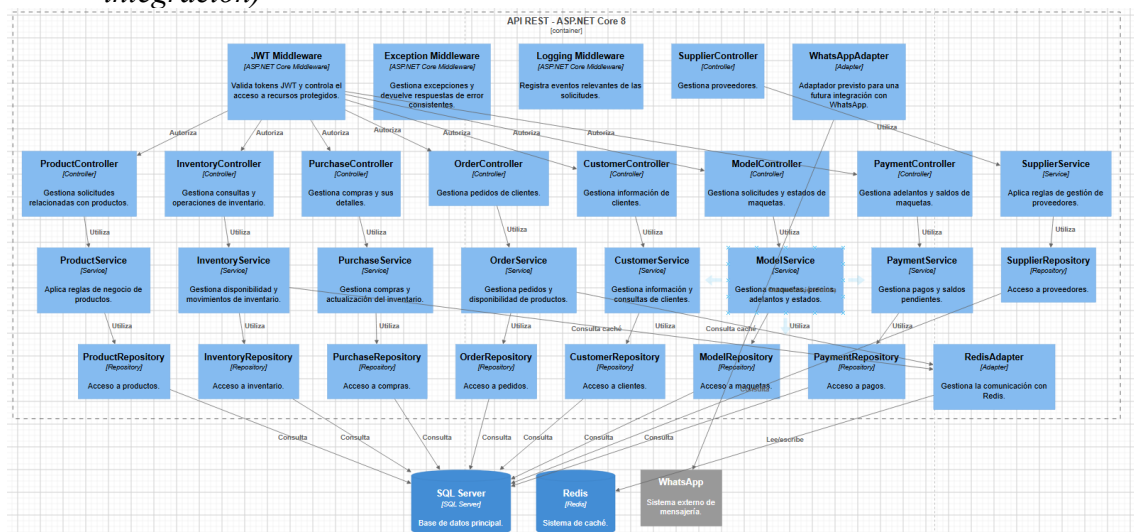
- ProductRepository
- InventoryRepository
- SupplierRepository
- PurchaseRepository
- OrderRepository
- CustomerRepository
- ModelRepository
- PaymentRepository

Middleware

- JwtMiddleware
- ExceptionMiddleware
- LoggingMiddleware

Adaptadores

- RedisAdapter
- WhatsAppAdapter *(previsto únicamente si posteriormente se confirma la integración)*



8. ADR-001 – Uso de una API REST centralizada

ID: ADR-001

Título: Uso de ASP.NET Core 8 como API REST centralizada

Estado: Aceptado

Contexto

El sistema contará con una aplicación web desarrollada en React y una aplicación móvil desarrollada en Flutter. Ambas aplicaciones necesitan acceder a las mismas funcionalidades y datos.

Se requiere evitar que cada aplicación implemente por separado las reglas de negocio y el acceso a la base de datos.

Alternativas consideradas

Alternativa 1 — Lógica independiente en cada aplicación

Cada cliente implementaría su propia lógica de negocio.

Alternativa 2 — Acceso directo a SQL Server

Las aplicaciones accederían directamente a la base de datos.

Alternativa 3 — API REST centralizada

Las aplicaciones se comunicarían con un backend que centralice las reglas de negocio y el acceso a datos.

Decisión

Se utilizará **ASP.NET Core 8 como API REST centralizada**, consumida mediante HTTPS por las aplicaciones React y Flutter.

Consecuencias

Positivas:

- La lógica de negocio estará centralizada.
- React y Flutter podrán utilizar los mismos servicios.
- La base de datos no estará expuesta directamente a los clientes.
- Facilita futuras integraciones.
- Permite implementar autenticación mediante JWT en un punto central.

Negativas:

- Es necesario desarrollar y mantener una API.
- Se debe gestionar autenticación, autorización y errores.
- Las aplicaciones dependen de la disponibilidad del backend.

9. ADR-002 SQL Server como persistencia y Redis como cache

ID: ADR-002

Título: Uso combinado de SQL Server y Redis

Estado: Aceptado

Contexto

El sistema necesita almacenar permanentemente información de productos, inventario, compras, pedidos, clientes, maquetas y pagos.

Además, algunas consultas relacionadas con productos y disponibilidad pueden realizarse frecuentemente.

Alternativas consideradas

Alternativa 1 — Utilizar únicamente SQL Server

Toda la información sería almacenada y consultada directamente desde SQL Server.

Alternativa 2 — Utilizar Redis como almacenamiento principal

Redis sería utilizado como fuente principal de información.

Alternativa 3 — SQL Server + Redis

SQL Server será la fuente principal de persistencia y Redis se utilizará como caché.

Decisión

Se utilizará **SQL Server como base de datos principal** y **Redis como mecanismo de caché** para información de consulta frecuente.

Consecuencias

Positivas:

SQL Server mantiene la información permanente.

Redis puede reducir consultas repetitivas.

Se puede mejorar el rendimiento de determinadas operaciones.

Se mantiene una separación clara entre persistencia y caché.

Negativas:

Se incorpora un componente adicional a la infraestructura.

Es necesario controlar la actualización e invalidación de la caché.

El equipo debe garantizar que la información almacenada temporalmente no contradiga los datos de SQL Server.

10.Relacion entre C4 y los requisitos

Para que el Documento 2 y el Documento 3 estén conectados:

Requisito	Componente C3 relacionado
RF-01 Gestión de productos	ProductController → ProductService → ProductRepository
RF-02 Control de inventario	InventoryController → InventoryService → InventoryRepository
RF-03 Proveedores	SupplierController → SupplierService → SupplierRepository
RF-04 Compras	PurchaseController → PurchaseService → PurchaseRepository
RF-05 Pedidos	OrderController → OrderService → OrderRepository
RF-06 Disponibilidad	InventoryService + RedisAdapter
RF-07 Registro de maquetas	ModelController → ModelService → ModelRepository
RF-08 Adelantos y saldos	PaymentController → PaymentService → PaymentRepository
RF-09 Estados de maquetas	ModelService
RF-10 Clientes	CustomerController → CustomerService → CustomerRepository
RNF-01 Rendimiento	RedisAdapter + API
RNF-02 Seguridad	JWT Middleware
RNF-03 Usabilidad	React / Flutter
RNF-04 Disponibilidad	ASP.NET Core 8 API
RNF-05 Mantenibilidad	Arquitectura Controller → Service → Repository